

Sampling and Monte Carlo checks

Simon Frost

Table of contents

1	Overview	1
2	Setup	2
3	The reference model and its exact marginals	2
4	Ancestral sampling	2
5	Convergence	3
6	Conditional queries by rejection	4
7	Interventions are respected by the sampler	5
8	Samples from <code>BayesianNetworks.jl</code>	5
9	A deterministic mechanism: asia	6
10	Summary	6
11	References	6

1 Overview

Variable elimination is exact, but it is only as trustworthy as its implementation. `BayesianNetworkInference.jl` therefore cross-checks it in two independent ways: against the brute-force joint table (`brute_force_marginal`, and `BayesianNetworks.marginal` at the model level), and against ancestral sampling, which never multiplies a factor. This vignette draws samples from the SPEC section 45 reference habitat model, watches the empirical marginals converge to the exact posteriors, and uses the same machinery for conditional and interventional queries.

2 Setup

```
using BayesianNetworks
using BayesianNetworkInference
using Random
```

3 The reference model and its exact marginals

`reference_habitat_model()` is the seven-variable ecological network of the SPEC with every conditional probability table bound. `infer` compiles it to a factor graph and eliminates; the result is a normalised Factor.

```
m = reference_habitat_model()
vars = variables(compile(m))
```

```
7-element Vector{Symbol}:
 :Climate
 :Irrigation
 :SoilMoisture
 :GrazingPressure
 :Vegetation
 :HabitatQuality
 :Occupancy
```

```
exact = Dict{v => infer(m, v)[1] for v in vars}
exact[:Occupancy].table
```

```
2-element Vector{Float64}:
 0.5237590625
 0.4762409375
```

4 Ancestral sampling

`ancestral_sample(m, n)` visits the variables in topological order and draws each from its mechanism's kernel given the sampled parents (SPEC section 18). The result is an `AncestralSamples` table with one column per variable; `empirical_marginal` turns a column (or several) into a factor with the same scope and axes as the exact posterior, so the two can be compared directly.

```
s = ancestral_sample(m, 20_000; rng=MersenneTwister(2026))
```

```
AncestralSamples: 20000 samples of (:Climate, :Irrigation, :SoilMoisture,
 :GrazingPressure, :Vegetation, :HabitatQuality, :Occupancy)
```

```
s[:Occupancy][1:8]
```

```
8-element Vector{Symbol}:
 :present
 :present
 :absent
 :absent
 :present
 :present
 :present
 :present
```

```
emp = empirical_marginal(s, :Occupancy)
emp.table, isapprox(emp, exact[:Occupancy]; atol=0.02)
```

```
([0.524, 0.476], true)
```

Joint frequencies work the same way, and the ordering of the requested variables is the scope of the result.

```
emp_joint = empirical_marginal(s, [:Vegetation, :Occupancy])
scope(emp_joint), maximum(abs, emp_joint.table .- infer(m, [:Vegetation,
:Occupancy]))[1].table)
```

```
([:Vegetation, :Occupancy], 0.00275718750000000357)
```

5 Convergence

The Monte Carlo error of a relative frequency scales as $1/\sqrt{n}$. The table below reports, for growing sample sizes, the largest absolute deviation between an empirical marginal and the exact one over all seven variables, next to $1/\sqrt{n}$ for scale. Every row uses a fresh seed so the rows are independent draws.

```
function max_error(samples)
    return maximum(maximum(abs, empirical_marginal(samples, v).table .-
        exact[v].table)
        for v in vars)
end

sizes = [100, 1_000, 10_000, 100_000]
rows = map(enumerate(sizes)) do (i, n)
    err = max_error(ancestral_sample(m, n; rng=MersenneTwister(100 + i)))
    (n=n, max_error=round(err; digits=4), scale=round(1 / sqrt(n); digits=4))
end
for r in rows
    println(lpad(r.n, 8), " ", lpad(r.max_error, 9), " ", lpad(r.scale, 8))
end
```

100	0.1111	0.1
1000	0.0217	0.0316
10000	0.0068	0.01
100000	0.0025	0.0032

The error falls by roughly a factor of three per tenfold increase in n , as expected, and the exact posterior never moves.

6 Conditional queries by rejection

Ancestral samples come from the prior (or interventional) distribution, never the posterior. Conditioning by rejection, keeping only the samples consistent with the evidence, is the crudest possible posterior estimate, but it is independent of every factor operation and therefore a useful check of infer with evidence.

```
keep = s[:Vegetation] .== :dense
kept = AncestralSamples(s.vars, s.axes, s.states[keep, :])
length(kept)
```

5177

```
posterior_mc = empirical_marginal(kept, :Occupancy)
posterior_ve, _ = infer(m, :Occupancy; evidence=Dict(:Vegetation => :dense))
posterior_mc.table, posterior_ve.table
```

([0.33513617925439443, 0.6648638207456056], [0.3325, 0.6675])

```
isapprox(posterior_mc, posterior_ve; atol=0.02)
```

true

posterior returns the same numbers as a dictionary keyed by state.

```
posterior(m, :Occupancy; evidence=Dict(:Vegetation => :dense))
```

```
Dict{Symbol, Float64} with 2 entries:
 :present => 0.6675
 :absent  => 0.3325
```

7 Interventions are respected by the sampler

A hard intervention rewrites the mechanism of a variable into a point mass, so both the sampler and the exact engine see the intervened model. Forcing low grazing pressure changes the occupancy distribution, and the samples agree with infer on the intervened model.

```
m_do = do_intervention(m, :GrazingPressure => :low)
s_do = ancestral_sample(m_do, 20_000; rng=MersenneTwister(3))
all(==( :low), s_do[:GrazingPressure])
```

```
true
```

```
empirical_marginal(s_do, :Occupancy).table, infer(m_do, :Occupancy)[1].table
```

```
([0.48365, 0.51635], [0.48833975, 0.51166025])
```

```
isapprox(empirical_marginal(s_do, :Occupancy), infer(m_do, :Occupancy)[1];
atol=0.02)
```

```
true
```

8 Samples from BayesianNetworks.jl

BayesianNetworks.sample(m, n) draws the same distribution as a vector of dictionaries. AncestralSamples(m, samples) converts that output so that it can be compared through the same empirical_marginal, and empirical_marginal(m, samples, var) does both steps at once.

```
bs = sample(m, 20_000; rng=MersenneTwister(4))
bs[1]
```

```
Dict{Symbol, Symbol} with 7 entries:
```

```
:Vegetation      => :dense
:GrazingPressure => :low
:HabitatQuality  => :good
:SoilMoisture    => :high
:Occupancy       => :present
:Climate         => :normal
:Irrigation      => :high
```

```
converted = AncestralSamples(m, bs)
empirical_marginal(m, bs, :Occupancy) == empirical_marginal(converted, :Occupancy)
```

```
true
```

```
isapprox(empirical_marginal(m, bs, :Occupancy), exact[:Occupancy]; atol=0.02)
```

```
true
```

9 A deterministic mechanism: asia

The either node of the asia network is a logical OR of lung and tub. Sampling respects a deterministic kernel exactly (no sample has lung = yes and either = no), and the sampled marginal of dysp agrees with the exact $P(\text{dysp} = \text{yes}) = 0.4360$.

```
asia = read_bayesnet(fixture_path("bif/asia.bif"))
sa = ancestral_sample(asia, 20_000; rng=MersenneTwister(5))
all(sa[:either][i] == :yes for i in 1:length(sa) if sa[:lung][i] == :yes)
```

```
true
```

```
empirical_marginal(sa, :dysp).table, infer(asia, :dysp)[1].table
```

```
([0.43625, 0.56375], [0.4359706, 0.5640294])
```

10 Summary

Ancestral sampling draws from the joint by walking the network in topological order, so a seeded run is reproducible, deterministic mechanisms are respected exactly, and an intervention is sampled as an intervention rather than as evidence. Empirical marginals converge to the exact ones at the expected Monte Carlo rate, which makes sampling a cheap independent check on the exact backends ([Koller and Friedman 2009](#)); rejection sampling answers conditional queries but pays for rare evidence. The next vignette, *Junction trees and belief propagation*, returns to exact inference and computes every marginal in a single pass.

11 References

Koller, Daphne, and Nir Friedman. 2009. *Probabilistic Graphical Models: Principles and Techniques*. MIT Press.